Task 08 - Debugging, Review and Evidence

Handbook reference: Page 8 of 10 | Report date: 2026-10-13 | Folder: `task-08-debugging-review-evidence/`

1. Concept from the handbook

Debugging means narrowing the cause with testable evidence and restoring intended behaviour: reproduce -> relevant error -> hypothesis -> focused change -> verify -> regression check. Do not call a failure 'flaky' without evidence, do not weaken assertions to get green, and remember that tests written with the same wrong assumption can both pass.

2. What was built

A real **bug** (UI label 'Done' vs stored 'done'/'DONE'). Tools to run the **same tests against the unchanged base and the fixed revision**, inspect the **data shape**, derive a hypothesis, classify failures as **regression vs pre-existing**, and render the **evidence-backed report** template (Change, Revision, Checks run, Results, Checks not run, Limitations, Next action).

3. Behaviours demonstrated

- Bug reproduces on base (tests FAIL), passes on the fixed revision
- Positive, negative and regression test cases
- Failure that also exists on base is labelled PRE-EXISTING, not a regression
- Report lists 'checks not run' honestly

4. Files in this task

`app/base/tracker.py`, `app/fixed/tracker.py`, `debug_tools.py`, `demo.py`, `test_debug_tools.py`, `test_filter.py`

Run it: `python demo.py` Test it: `python -m unittest discover`

5. Code screenshots

debug_tools.py

```python
"""Task 08 - Debugging, review and evidence.

reproduce -> relevant error -> hypothesis -> focused change -> verify -> regression check
"""
from __future__ import annotations

import hashlib
import os
import subprocess
import sys
from collections import Counter
from dataclasses import dataclass, field
from pathlib import Path
from typing import Dict, List

HERE = Path(__file__).parent


def run_tests(revision_dir: Path, pattern: str = "test_filter.py") -> dict:
    """Run the SAME tests against a given revision of the code."""
    env = dict(os.environ, TRACKER_DIR=str(revision_dir))
    cp = subprocess.run([sys.executable, "-m", "unittest", pattern, "-v"], cwd=HERE, env=env,
                        capture_output=True, text=True)
    text = cp.stdout + cp.stderr
    failed = [l.split(" ")[0] for l in text.splitlines() if l.endswith("... FAIL") or l.endswith("... ERROR")]
    return {"rc": cp.returncode, "failed": failed, "text": text}


def inspect_data_shape(tasks: List[dict]) -> Dict[str, int]:
    """Don't treat the error as the cause: look at the real data first."""
    return dict(Counter(t["status"] for t in tasks))


def hypothesis_from_shape(shape: Dict[str, int], ui_labels: List[str]) -> str:
    hits = [(lbl, s) for lbl in ui_labels for s in shape if s != lbl and s.lower() == lbl.lower()]
    if hits:
        lbl, s = hits[0]
        return f"UI label '{lbl}' != stored value '{s}' (case mismatch) -> normalise before comparing"
    return "no case-mismatch found; keep investigating (do not guess)"


def revision_hash(path: Path) -> str:
    return hashlib.sha256((path / "tracker.py").read_bytes()).hexdigest()[:10]


@dataclass
class EvidenceReport:
    """Handbook template: Change / Revision / Checks run / Results / Not run / Limitations / Next."""
    change: str
    revision: str
    checks_run: List[str]
    results: str
    checks_not_run: List[str]
    limitations: List[str]
    next_action: str

    def render(self) -> str:
        rows = [("Change", self.change), ("Revision tested", self.revision),
                ("Checks run", "; ".join(self.checks_run)), ("Results", self.results),
                ("Checks not run", "; ".join(self.checks_not_run) or "none"),
                ("Known limitations", "; ".join(self.limitations) or "none stated"),
                ("Next required action", self.next_action)]
        return "\n".join(f"{k + ':':<22}{v}" for k, v in rows)


def classify_failure(base_result: dict, new_result: dict) -> str:
    """If the failure also happens on the unchanged base, it is NOT a regression from our change."""
    new_f, base_f = set(new_result["failed"]), set(base_result["failed"])
    if not new_f:
        return "no failures on new revision"
    same = new_f & base_f
    regress = new_f - base_f
    parts = []
    if regress:
        parts.append(f"REGRESSION (new failures): {sorted(regress)}")
    if same:
        parts.append(f"PRE-EXISTING (also fail on base): {sorted(same)}")
    return "; ".join(parts)
```

Main implementation: debug_tools.py (part 1/2)

Main implementation: debug_tools.py (part 2/2)

demo.py

```python
from debug_tools import (EvidenceReport, HERE, classify_failure, hypothesis_from_shape,
                         inspect_data_shape, revision_hash, run_tests)

BASE, FIXED = HERE / "app" / "base", HERE / "app" / "fixed"
DATA = [{"id": 1, "status": "open"}, {"id": 2, "status": "done"}, {"id": 3, "status": "Open"}, {"id": 4, "status": "DON

print("1) REPRODUCE on the unchanged base")
base = run_tests(BASE)
print(f"   rc={base['rc']}  failing: {base['failed']}")
print("   relevant error line:", [l for l in base['text'].splitlines() if 'AssertionError' in l][:1])

print("\n2) INSPECT DATA SHAPE (error is not the cause)")
shape = inspect_data_shape(DATA)
print("   stored statuses:", shape)
print("   hypothesis    :", hypothesis_from_shape(shape, ["Done", "Open"]))

print("\n3) FOCUSED CHANGE -> 4) VERIFY (positive + negative + regression cases)")
new = run_tests(FIXED)
print(f"   rc={new['rc']}  failing: {new['failed'] or 'none'}")
print("    summary:", [l for l in new["text"].splitlines() if l.startswith("Ran")][0], "-> OK")

print("\n5) REGRESSION CHECK vs unchanged base")
print("   ", classify_failure(base, new))

print("\n6) EVIDENCE-BACKED REPORT")
print(EvidenceReport(
    change="Status filter normalization (case-insensitive compare)",
    revision=f"base={revision_hash(BASE)} -> fixed={revision_hash(FIXED)} (content hashes)",
    checks_run=["python -m unittest test_filter.py -v (5 cases: positive, negative, regression)"],
    results=f"base: FAIL ({len(base['failed'])} cases)  fixed: PASS (rc={new['rc']})",
    checks_not_run=["browser/UI test - no UI in this lab", "hosted CI - no remote"],
    limitations=["only covers case differences; whitespace handled but other status vocab untested"],
    next_action="human review of diff",
).render())
```

Demo script

6. Demo and test screenshots

terminal - run the tests

```
$ python -m unittest discover -v
test_bug_reproduces_on_base (test_debug_tools.DebugToolsTests.test_bug_reproduces_on_base) ... ok
test_data_shape_and_hypothesis (test_debug_tools.DebugToolsTests.test_data_shape_and_hypothesis) ... ok
test_fix_passes_all_cases (test_debug_tools.DebugToolsTests.test_fix_passes_all_cases) ... ok
test_regression_check_separates_old_and_new_failures
(test_debug_tools.DebugToolsTests.test_regression_check_separates_old_and_new_failures) ... ok
test_report_has_all_template_fields (test_debug_tools.DebugToolsTests.test_report_has_all_template_fields) ... ok
test_revision_hash_differs (test_debug_tools.DebugToolsTests.test_revision_hash_differs) ... ok
test_all_returns_everything_in_order (test_filter.FilterTests.test_all_returns_everything_in_order) ... ok
test_done_label_matches_stored_lowercase (test_filter.FilterTests.test_done_label_matches_stored_lowercase) ... ok
test_done_never_contains_open (test_filter.FilterTests.test_done_never_contains_open) ... ok
test_open_returns_open_in_any_case (test_filter.FilterTests.test_open_returns_open_in_any_case) ... ok
test_unknown_status_returns_nothing (test_filter.FilterTests.test_unknown_status_returns_nothing) ... ok

----------------------------------------------------------------------
Ran 11 tests in 0.290s

OK
```

Test run (real output)

terminal - python demo.py

```
$ python demo.py
1) REPRODUCE on the unchanged base
   rc=1  failing: ['test_done_label_matches_stored_lowercase', 'test_open_returns_open_in_any_case']
   relevant error line: ['AssertionError: Lists differ: [] != [2, 4]']

2) INSPECT DATA SHAPE (error is not the cause)
   stored statuses: {'open': 1, 'done': 1, 'Open': 1, 'DONE': 1}
   hypothesis    : UI label 'Done' != stored value 'done' (case mismatch) -> normalise before comparing

3) FOCUSED CHANGE -> 4) VERIFY (positive + negative + regression cases)
   rc=0  failing: none
    summary: Ran 5 tests in 0.000s -> OK

5) REGRESSION CHECK vs unchanged base
    no failures on new revision

6) EVIDENCE-BACKED REPORT
Change:               Status filter normalization (case-insensitive compare)
Revision tested:      base=d39b627d5c -> fixed=34b62c289b (content hashes)
Checks run:           python -m unittest test_filter.py -v (5 cases: positive, negative, regression)
Results:              base: FAIL (2 cases)  fixed: PASS (rc=0)
Checks not run:       browser/UI test - no UI in this lab; hosted CI - no remote
Known limitations:    only covers case differences; whitespace handled but other status vocab untested
Next required action: human review of diff
```

Demo output (python demo.py)

7. Evidence-backed report

Change	Debugging, Review and Evidence
Revision tested	content hash `889c1bbd0b53` of all .py files in this folder (re-run with the git commit SHA after you commit)
Checks run	`python -m unittest discover` and `python demo.py`
Results	**PASS** - Ran 11 tests in 0.246s
Checks not run	Browser/UI tests; Hosted CI
Known limitations	Only case/whitespace differences in status are covered.
Next required action	Human review, then CI on the pull request

8. Practical takeaway

Passing tests are evidence, not proof of everything. Only behaviour the checks cover is supported.